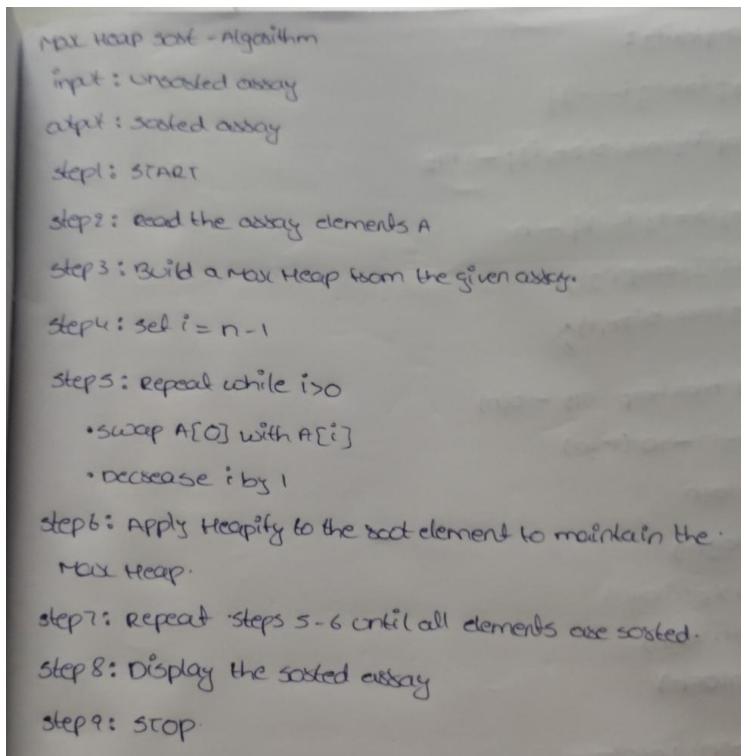


Practical-03

Aim: Implementation of max-heap sort algorithm.

1.) Max Heap

Algorithm:



Program:

```
#include <iostream>
```

```
using namespace std;
```

```
void maxheapify(int arr[], int size, int i)
```

```
{
```

```
    int largest = i;
```

```
    int l = 2 * i;
```

```

int r = 2 * i + 1;

if (l <= size && arr[l] > arr[largest])
{
    largest = l;
}

if (r <= size && arr[r] > arr[largest])
{
    largest = r;
}

if (largest != i)
{
    swap(arr[largest], arr[i]);
    maxheapify(arr, size, largest);
}
}

void heapsort(int arr[], int size)
{
    for (int i = size / 2; i >= 1; i--)
    {
        maxheapify(arr, size, i);
    }
    for (int i = size; i > 1; i--)
    {
        swap(arr[1], arr[i]);
        maxheapify(arr, i - 1, 1);
    }
}

int main()
{
    int n;

    cout << "Enter number of elements: ";

    cin >> n;

```

```
int arr[n + 1];

cout << "Enter elements: ";

for (int i = 1; i <= n; i++)
{
    cin >> arr[i];
}

heapsort(arr, n);

cout << "Sorted array: ";

for (int i = 1; i <= n; i++)
{
    cout << arr[i] << " ";
}

return 0;
}
```

Output:

```
Output
Enter number of elements: 5
Enter elements: 5
9
7
6
1
Sorted array: 1 5 6 7 9

=== Code Execution Successful ===
```

Time Complexity:

time complexity :
 for ($i = n/2$; $i > 1$; $i--$) {
 maxHeapify(arr, size, i); $\rightarrow n/2$
 }
 for ($i = size$; $i > 1$; $i--$) { $\rightarrow n$
 swap(arr[i], arr[1]);
 maxHeapify(arr, size, 1);
 }
 Building the maxHeap $\rightarrow n/2 \rightarrow O(n)$
 maxHeapify $\rightarrow O(\log n)$
 $\therefore n \times \log n$
 $= O(n \log n)$
 $= O(n) \times O(\log n)$
 $= O(n \log n)$
 $T(n) = O(n \log n)$
 Best case $= O(n) + O(n \log n)$
 $= O(n \log n)$
 Average case $= O(n) + O(n \log n)$
 $= O(n \log n)$
 Worst case $= O(n) + O(n \log n)$
 $= O(n \log n)$

Conclusion:

Max-Heap Sort efficiently sorts elements using a max heap. It has a time complexity of $O(n \log n)$ and works well for sorting large arrays.